

Object Oriented Programming

OOPs = Object oriented programming structure

OOPs is a programming model or paradigm which revolves around the concept of "OBJECTS".

An **object** refers to the instance of the class, which contains the instance of the members and behaviors defined in the class template. In the real world, an object is an actual entity to which a user interacts, whereas class is just the blueprint for that object. So the objects consume space and have some characteristic behavior.

A **class** can be understood as a template or a blueprint, which contains some values, known as member data or member, and some set of rules, known as behaviors or functions. So when an object is created, it automatically takes the data and functions that are defined in the class.

It deals with internal designing not external. So it is just to help the programmer to prevent data from mishandling.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.

There are several benefits to using object-oriented programming (OOP) in software development:

Modularity: OOP allows you to break a large and complex problem down into smaller and more manageable pieces, or "objects." This makes it easier to write, test, and maintain your code.

Reusability: OOP allows you to reuse code by creating new objects that are similar to existing ones. This can save time and effort when developing software.

Extensibility: OOP allows you to easily add new features to an existing codebase by creating new objects or modifying existing ones.

Maintainability: OOP makes it easier to track and fix errors in your code, as objects are self-contained and can be isolated and tested more easily than procedural code.

Scalability: OOP allows you to build software that can grow and evolve over time, as new objects can be created or added to the existing codebase as needed.

Oops, are based on a bottom-up approach, unlike the structural programming paradigm, which uses a top-down approach.

What are some other programming paradigms other than OOPs?

Programming paradigms refers to the method of classification of programming languages based on their features. There are mainly two types of Programming Paradigms:

Imperative Programming Paradigm

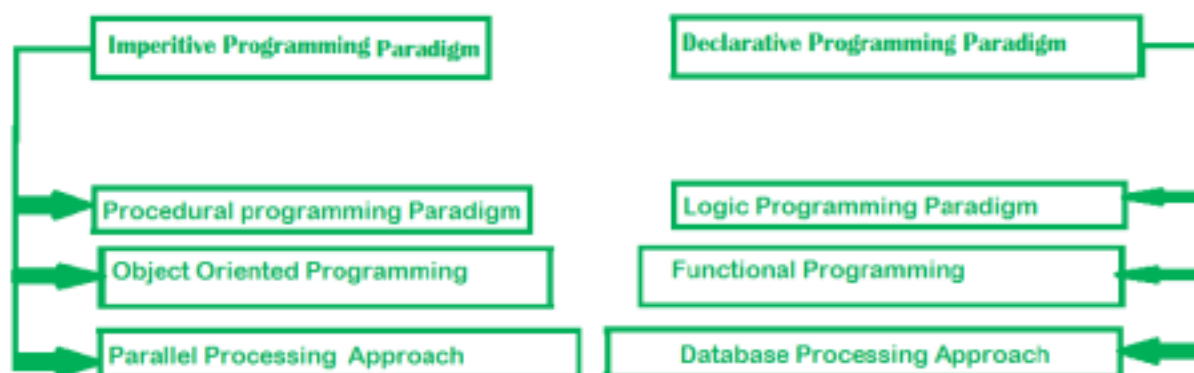
Declarative Programming Paradigm

Now, these paradigms can be further classified based: 1.

Imperative Programming Paradigm: Imperative programming focuses on HOW to execute program logic and defines control flow as statements that change a program state. This can be further classified as:

- a) Procedural Programming Paradigm:** Procedural programming specifies the steps a program must take to reach the desired state, usually read in order from top to bottom.
- b) Object-Oriented Programming or OOP:** Object-oriented programming (OOP) organizes programs as objects that contain some data and have some behavior.
- c) Parallel Programming:** Parallel programming paradigm breaks a task into subtasks and focuses on executing them simultaneously at the same time.
- 2. Declarative Programming Paradigm:** Declarative programming focuses on WHAT to execute and defines program logic, but not a detailed control flow. Declarative paradigm can be further classified into:
- a) Logical Programming Paradigm:** Logical programming paradigm is based on formal logic, which refers to a set of sentences expressing facts and rules about how to solve a problem
- b) Functional Programming Paradigm:** Functional programming is a programming paradigm where programs are constructed by applying and composing functions.
- c) Database Programming Paradigm:** Database programming model is used to manage data and information structured as fields, records, and files. be created or added to the existing codebase as needed.

Programming Paradigms



Why do we need a Constructor in Oops?

Creating an object without a constructor is like buying a car with no color or we will color it after buying. Using Accessor and Mutators functions is philosophically wrong. Therefore the constructor must be there for initializing the object. At the time of placing the order I wanted a white color Scorpio or fortuner etc.

So the purpose of a constructor is to create a new object and set values for any existing object properties.

A constructor in C++ is a special method that is automatically called when an object of a class is created. To create a constructor, use the same name as the class, followed by parentheses ().

There should not be a return type of constructor.

```

1 //Accessor and Mutator
2 #include<bits/stdc++.h>
3 using namespace std;
4 class Marker{
5 private:
6     string color;
7     string nature;
8 public:
9     //Default Constructor or Non - parameterized Constructor
10    Marker(){
11        color = "Red";
12        nature = "Temporary";
13    }
14    //Parameterized Constructor
15    Marker(string color = "Red", string nature = "Temporary"){
16        this->color = color;
17        this->nature = nature;
18    }
19    //Copy Constructor
20    Marker(Marker &m){
21        this->color = m.color;
22        this->nature = m.nature;
23    }
24    pair<string ,string> getter(){
25        return {color,nature};
26    }
27 };
28 int main(){
29     Marker m1("Blue","Temporary"), m2("Black","Permanent");
30     //Marker m3;
31     //When we calling m3 without passing any thing then
32     //Error - call of overloaded 'Marker()' is ambiguous, so either comment default constructor
33     //Or Pass Atleast One Arguments
34     Marker m3(m2);
35     cout<<"Marker M1 - Color "<<m1.getter().first<<" Nature "<<m1.getter().second<<endl;
36     cout<<"Marker M2 - Color "<<m2.getter().first<<" Nature "<<m2.getter().second<<endl;
37     cout<<"Marker M3 - Color "<<m3.getter().first<<" Nature "<<m3.getter().second;
38     return 0;
39 }

```

```
≡ output.txt X
≡ output.txt
1  Marker M1 - Color Blue Nature Temporary
2  Marker M2 - Color Black Nature Permanent
3  Marker M3 - Color Black Nature Permanent
```

Deep Copy Constructor :-

```
1 //Deep Copy Constructor
2 #include<bits/stdc++.h>
3 using namespace std;
4 class Test{
5 private:
6     int a;
7     int *p;
8 public:
9     Test(int x){
10         a = x;
11         p = new int[5];
12     }
13     Test(Test &t){
14         this->a = t.a;
15         //this->p = t.p; // This is wrong.
16         this->p = new int[5]; //here we called deep copy constructor.
17     }
18 };
```

There are three types of constructors:

- Default / Non Parameterized Constructor - With no parameters.
C++ says that a default constructor is automatically provided by the compiler.
- Parameterized Constructor - With Parameters, Create a new instance of a class and also pass arguments simultaneously.
- Copy Constructor - Which creates a new object as a copy of an existing object.

Type of Function inside Class :-

- Constructor
- Accessor
- Mutator
- Facilitators
- Enquiry/Inspector
- Destructor

The using Directive :-

A C++ program can be divided into different namespaces. A namespace is a part of the program in which certain names are recognized; outside of the namespace they're unknown.

The directive **using namespace std;**

says that all the program statements that follow are within the std namespace. Various program components such as cout are declared within this namespace.

If we didn't use the using directive, we would need to add the std name to many program elements.

For example, in the FIRST program we'd need to say

std::cout << "Every age has a language of its own.";

Scope Resolution Operators (::) - it is used for the following purposes.

1. To access a global variable when there is a local variable with same name:



```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int x = 4;
4  int main(){
5      int x = 10;
6      cout<<"Value of global x is "<<::x;
7      cout<<"\nValue of local x is "<<x;
8      return 0;
9  }
10
11 Output :
12 Value of global x is 4
13 Value of local x is 10
```

2.To Define a Function Outside the class.


```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class A{
4  private:
5      int a,b;
6  public:
7      A(int x, int y){
8          a=x,b=y;
9      }
10     int Multiplication();
11 };
12 int A::Multiplication(){
13     return a*b;
14 }
15 int main(){
16     A obj(5,6);
17     cout<<"The Multiplication is : "<<obj.Multiplication();
18     return 0;
19 }
20
21 Output :
22 The Multiplication is : 30
```

3.To access a class's static variables.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4      static int x;
5  public:
6      static int y;
7      void func(int x){
8          cout << "Value of static x is " << Test::x;
9          cout << "\nValue of local x is " << x;
10     }
11 };
12 // Static Member we can access directly without creating object.
13 int Test::x = 1;
14 int Test::y = 2;
15
16 int main(){
17     Test obj;
18     int x = 3 ;
19     obj.func(x);
20     cout << "\nTest::y = " << Test::y;
21     return 0;
22 }
23
24 Output:
25 Value of static x is 1
26 Value of local x is 3
27 Test::y = 2
28

```

4. In Case of multiple Inheritance: if the same variable name exists in two ancestor classes, we can use scope resolution operator to distinguish.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class A{
4  protected:
5      int x;
6  public:
7      A() { x = 10; }
8  };
9  class B{
10 protected:
11     int x;
12 public:
13     B() { x = 20; }
14 };
15
16 class C: public A, public B{
17 public:
18     void fun(){
19         cout << "A's x is " << A::x;
20         cout << "\nB's x is " << B::x;
21     }
22 };
23 int main(){
24     C c;
25     c.fun();
26     return 0;
27 }
28
29 Output:
30 A's x is 10
31 B's x is 20
```

5.Refer to a class inside another class :- refer to a nested class using the scope resolution operator.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class outside{
4  public:
5      int x;
6      class inside {
7      public:
8          int x;
9          static int y;
10         int foo();
11     };
12 };
13 int outside::inside::y = 5;
14 int main(){
15     outside A;
16     outside::inside B;
17 }
18

```

Inline Function :-

When a function is declared inline, the compiler places a copy of that specific function's code at each point where the function is called at compile time.

When the function is declared outside the class with inline references then this can become overhead if the execution time of function is less than the switching time from the caller function to called function (callee).

Note :- Inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining.

Compiler may not perform inlining in such circumstances:

- If the function contains a loop.
- If a function contains static Variables
- If a function is recursive in nature.
- If a function return type is other than void
- If a function contains switch or goto statements.

```
1  #include <iostream>
2  using namespace std;
3  class operation
4  {
5      int a,b,add,sub,mul;
6      float div;
7  public:
8      void get();
9      void sum();
10     void difference();
11 };
12 inline void operation :: get(){
13     cout << "Enter first value:";
14     cin >> a;
15     cout << "Enter second value:";
16     cin >> b;
17 }
18 inline void operation :: sum(){
19     add = a+b;
20     cout << "Addition of two numbers: " << a+b << "\n";
21 }
22 inline void operation :: difference(){
23     sub = a-b;
24     cout << "Difference of two numbers: " << a-b << "\n";
25 }
26 int main(){
27     cout << "Program using inline function\n";
28     operation s;
29     s.get();
30     s.sum();
31     s.difference();
32     return 0;
33 }
34
```

This Pointer :- this is a keyword that refers to the current instance of the class.

There can be 3 main uses of "this" keyword :

- It can be used to declare indexers.
- It can be used to refer to the current class instance variable.
- It can be used to pass the current object as a parameter to another method.



```
1 #include <iostream>
2 using namespace std;
3 class Node{
4 private:
5     int data;
6     Node* next;
7 public:
8     Node(int x){
9         this->data = x;
10        this->next = NULL;
11    }
12 };
13 int main(){
14     Node* n = new Node(5);
15     return 0;
16 }
```

Structure Class

A Structure is defined with the struct keyword

A class is defined with the class keyword

All the member of a structure are public by default	All the members of a class are private by default
Structure can't be inherit	Class can be inherit
A structure contains only data member	A class contain both data member and member function
There is no data hiding features	Classes having data hiding features by using access specifiers(public, private, protected)

Operator Overloading :- we can make operators work for user-defined classes. This means C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading.

For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +. Other

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;

```

example classes where arithmetic operators may be overloaded are Complex Numbers, Fractional Numbers, Big Integer, etc.


```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;
11     }
12     void display(){
13         cout << real << " + i" << img << endl;
14     }
15     Complex operator +(Complex c){
16         Complex temp;
17         temp.real = real + c.real;
18         temp.img = img + c.img;
19         return temp;
20     }
21     Complex operator -(Complex c){
22         Complex temp;
23         temp.real = real - c.real;
24         temp.img = img - c.img;
25         return temp;
26     }
27 };
28 int main(){
29     Complex c1(5, 3), c2(10, 5), c3, c4;
30     c3 = c2 + c1;
31     c4 = c2 - c1;
32     c4.display();
33     c3.display();
34 }
35
36 //Output
37 5 + i2
38 15 + i8

```

Operator Overloading using friend function :-

```

1  // Operator Overloading using Friend Functions
2  #include <iostream>
3  using namespace std;
4  class Complex{
5  private:
6      int real;
7      int img;
8  public:
9      Complex(int r = 0, int i = 0){
10         real = r;
11         img = i;
12     }
13     void display(){
14         cout << real << " + i" << img << endl;
15     }
16     friend Complex operator+(Complex c1, Complex c2);
17 };
18 Complex operator+(Complex c1, Complex c2){
19     Complex temp;
20     temp.real = c1.real + c2.real;
21     temp.img = c1.img + c2.img;
22     return temp;
23 }
24 int main(){
25     Complex c1(5, 3), c2(10, 6), c3;
26     c3 = c1 + c2;
27     // also written like this
28     // c3 = operator+(c1,c2);
29     c3.display();
30 }
31
32 // Output
33 15 + i9;

```

Ostream Operators Overloading :-



Eg - Addition of Rational Numbers



Inheritance

Inheritance :- It is one of the most important features of Object-Oriented Programming. The Capability of a class to derive properties and characteristics from another class is called inheritance.

Inheritance is a feature or a process in which, new classes are created from the existing classes. The new class created is called "Derived Class" or "Child Class" or "Sub Class" and the existing class is known as the "Base Class" or "Parent Class" or "Super Class".

When we say derived class inherits the base class, it means the derived class inherits all the properties of base class without changing the properties of base class and may add new features to its own derived class.

Why and when to use Inheritance.

- Inheritance is a term for reusing code by a mechanism of passing down information and behavior from a parent class to a child or subclass.
- Consider a group of vehicles. You need to create classes for Bus, Car, and Truck. The methods fuelAmount(), capacity(), applyBrakes(), will be the same for all three classes. If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes as shown below figure:-



- We can clearly see that the above process results in duplication of the same code 3 times. This increases the chances of error and data redundancy. To avoid this type of situation, inheritance is used.



- If you want the object to use all the behavior of the base class unless explicitly overridden, then inheritance is the simplest, least verbose, most straightforward way to express it.
- The main reason for using inheritance is not as a form of composition - it is so you can get polymorphic behavior. If you don't need polymorphism, you probably should not be using inheritance, at least in C++.

Available vs Accessible

Available : If a class Derived is inheriting from Base class then everything from Base class will be available/present in Derived class.

Accessible: everything is available in Derived class but private members are not accessible.

Example: bank balance of Father is available for Son by not accessible. Son has to ask Father for money. Only Father can access his account.



Constructors in inheritance :-

- Constructors of base class are executed first then the constructor of derived class is executed.
- By default, a non-parameterised constructor of base class is

executed.

- A parameterised constructor of base class must be called from derived class constructor.



The Diamond Problem (Multiple-path inheritance) :- The Diamond problem occurs when a child class inherits from two parent classes who both share a common grandparent class.



As shown in the figure, class *Child* inherits the traits of class *Person* twice—once from *Father* and again from *Mother*. This gives rise to ambiguity since the compiler fails to understand which way to go.



The solution to the diamond problem is to use the virtual keyword. We make the two parent classes (who inherit from the same grandparent class) into virtual classes in order to avoid two copies of the

grandparent class in the child class



Uses-A, IsA and HasA Relationship :-



Types Of Inheritance :-



1) Single Inheritance :- In single inheritance, a class derives from one base class only. This means that there is only one subclass that is derived from one superclass.



2) Multiple Inheritance :- Multiple inheritance is a type of inheritance in which a class derives from more than one class. As shown in the above diagram, class C is a subclass that has class A and class B as its parent.



Example :- In a real-life scenario, a child inherits from their father and mother. This can be considered an example of multiple inheritance.

3) Multilevel Inheritance :- In multilevel inheritance, a class is derived from another derived class. This inheritance can have as many levels as long as our implementation doesn't go wayward. In the above diagram, class C is derived from Class B. Class B is in turn derived from class A.



4) Hierarchical Inheritance :- In hierarchical inheritance, more than one class inherits from a single base class as shown in the representation above. This gives it a structure of a hierarchy.



5) Hybrid Inheritance :- Hybrid inheritance is usually a combination of more than one type of inheritance. In the above representation, we have multiple inheritance (B, C, and D) and multilevel inheritance (A, B, and D) to get a hybrid inheritance.



Access Specifier in Inheritance





Program to Demonstrate inheritance



Base Class Pointer and Derived Class Object in C++:-

```
Base * ptr;  
Ptr = new Derived();
```



Eg. Suppose there is a basic Car. A basic car means having the basic features of a car. Nothing is automated, no extra features, except for driving a car there are no extra things like there is no air conditioner, no media player, and no keyless entry. Nothing is there. Then you have an advanced car. And an advanced car is inheriting from a basic car. So, can we have a pointer of a basic car, and to that pointer, can we assign an object of an advanced car? Yes.

Then using the basic class pointer which functions you can call? You can call only the functions which are present in the basic car. It is just like a basic car but there is an advanced car here with all the extra features in the car which you have no idea about.



Can we create a Derived pointer assigned to the Base class object in C++?

```
Derived *p;  
p = new Base();
```

Is it possible that we have a derived class pointer and I have assigned the object of the base class? **No, not possible.** Why? Let us see the reason.

See we have a basic car. **Now if we are calling the basic car as an advanced car.** Then can we get the advanced car features in the

basic car? The answer is No.

So, many of the features of our advanced car are not available in the basic car. If I think that it is an advanced car, I cannot use the features. Suppose if I tried keyless entry then it is not there, if I try to open moon roof then it's not there, if I try to start AC then it's not there.

Polymorphism

Polymorphism is considered one of the important features of Object-Oriented Programming. The word polymorphism means having many forms. In simple words, **we can define polymorphism as the ability of a message to be displayed in more than one form.**

One real-life example of polymorphism is, that a person at the same time can have different characteristics. **A man at the same time is a father, a husband, and an employee.** So, the same person possesses different behavior in different situations. This is called polymorphism.



Compile Time Polymorphism in C++ :- This type of polymorphism is achieved by function overloading or operator overloading. The overloaded functions are invoked by matching the type and number of arguments.

The information is present during compile-time. This means the C++ compiler will select the right function at compile time. Which is also known as static binding or early binding.

Runtime Polymorphism in C++ :- Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

This type of polymorphism is achieved by Function Overriding. This happens when an object's method is invoked/called during runtime rather than during compile time.



Function overriding -



Key Points about Function overriding :-

- Redefining a function of a base class in the derived class called function overriding in c++.
- Function overriding is used for achieving runtime polymorphism.
- The Signature and Prototype of an overrides function must be exactly the same as the base function.

When do we need to override a function in C++?

If the superclass function logic is not fulfilling the sub-class business requirements, then the subclass needs to override that function with the required business logic. Usually, in most real-time applications, the superclass functions are implemented with generic logic which is common for all the next-level sub-classes.

Note :- If a function in the sub-class contains the same signature as the superclass non-private function, then the subclass function is treated as the **overriding function** and the superclass function is treated as the **overridden function**.

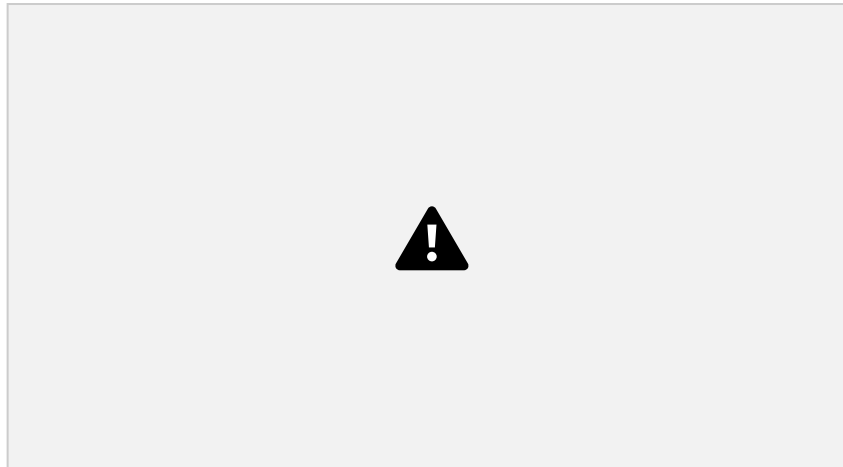
Virtual Function in C++ :- A virtual function in C++ is a member function that is declared within a base class using the virtual keyword and is re-defined by a derived class. When we refer to a **derived class object using the base class pointer** or base class reference variable, and when we can call a virtual function, then it will execute the function from the derived class.

So, Virtual Functions in C++ ensure that the correct function is called for an object, regardless of the expression used to make the function call.



Real-Time Example to Understand the need for Virtual

Function :



Pointer is the basic car but the actual object is the advanced car. So, I am thinking of it as a basic car. How can I drive? How it will run? Is it will run like what I am thinking or run like an advanced car? It will run like an advanced car because I am pointing at the object of an advanced car.

So, when you call a function that is present in the base case as well as the derived class, whose function must be called? Based on the object the function must be called. Not based on the pointer.

See it is just like if you saw a donkey and say it's a horse, will it run like a horse? You are thinking of it like a horse but that's a donkey. It cannot run like a horse. In the same way, here the basic car is the reference and you are pointing at the advanced car object. So, the object is of the derived class and the pointer is of the base class.

When we call a function then the base class function is called. But logically it is wrong. Whose function must be called? The function must be called based on the object.

Key points about virtual functions :-

1. Virtual functions are used for achieving polymorphism
2. The base class can have virtual functions
3. Virtual functions **can be** overrides in the derived class
4. Pure virtual functions **must be** overrides by the derived class

Rules for virtual functions :-

1. Virtual functions in C++ cannot be static.
2. A virtual function in C++ can also be a friend function of another class.
3. The prototype/Signature of virtual functions should be the same in the base as well as the derived class.
4. In C++, a class may have a virtual destructor but it cannot have a virtual constructor.

Limitation of virtual functions :-

- Slower
- Difficult to debug



Runtime Polymorphism :-

Now, suppose we want the derived class function should be called then we need to make the base class functions as virtual functions. This means if we want the Innova class Start function should be



called then we have to make the Car class Start function virtual.

Key Points of Runtime Polymorphism in C++:

- Same name different actions
- Runtime Polymorphism is achieved using function overriding
- Virtual functions are abstract functions of the base class
- The derived class must override the virtual functions
- A base class pointer pointing to a derived class object and an override function

is called

Abstract class in C++

An abstract class in C++ has at least one pure virtual function by definition. In other words, a function that has no definition and these classes cannot be instantiated. The abstract class's child classes must provide body to the pure virtual function; otherwise, the child class would become an abstract class in its own right.

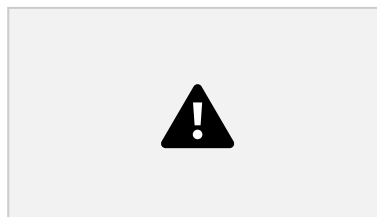
What is the purpose of the Pure Virtual function?

The purpose of a pure virtual function is to achieve polymorphism. We want the derived classes to override this function. So, it becomes mandatory for derived classes to override the pure virtual function. It means the base class is governing or giving a command to the child classes that you must override the pure virtual functions. So, it is defining an interface.

If a class is having a pure virtual function, then that class is called an **Abstract Class in C++**.

Can we create the object of Abstract Class in C++? No.

Can we create the pointer of the abstract class? Yes.



What is the purpose of inheritance?

- Two things, one is reusability. The derived class gets the function from the base class.
- The second thing to achieve is polymorphism.



Key Points of Abstract class in C++:

- The class having a pure virtual function is an abstract class.
- An **Abstract class** can have concrete also.
- The object of abstract class cannot be created
- A derived class must override pure virtual function, otherwise, it will also become an abstract class.
- The pointer of the abstract class can be created. • The pointer of the abstract class can hold the object of the derived class.
- Abstract classes are used for achieving polymorphism • Abstract with some concrete and some pure virtual functions.





Friend Function and Friend Classes in C++

A friend function in C++ is defined as a function that can access private, protected, and public members of a class. A friend function can be a member of another class or can be a global function in C++.

Note :- A friend function of a class is defined outside that class scope but it has the right to access all private, protected, and public members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

Characteristics of a Friend Function in C++:

- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object. - It cannot access the member names directly and has to use an object name and dot membership operator with the member's name. - It can be declared either in the private or the public part. - The function is not in the scope of the class of which it has been declared as a friend



Friend Classes in C++:

In C++, a friend class can access private, protected, and public members of another class in which it is declared a friend. It is sometimes useful to allow a particular class to access private members of other classes.

Instead of writing the name before the My Class definition, we can also write `friend class Your;`



Key Points of Friend Functions and Classes in C++ :-

- Friend functions are global functions
- They can access private, protected, and public members of a class upon their objects
- A class can be declared as a friend of another class
- All the functions of the friend class can access private and protected members of other classes.
- **Friendship is not mutual.** If class A is a friend of B, then B doesn't become a friend of A automatically.
- **Friendship is not inherited.**
- Friends should be used only for a limited purpose.

- Too many functions or external classes are declared as friends of a class with protected or private data, which is against the principle of encapsulation in object-oriented programming.

Static Data Members in C++ :-

We can define class members' static using the static keyword in C++. When we declare a member of a class as static **it means no matter how many objects of the class are created, there is only one copy of the static member available throughout the program. So static variables or static data members of a class belong to a class. That doesn't belong to an object.**

It means when we make a variable as static, that variable will occupy memory only once. And all the objects can share it.



One more thing we have to address syntactically. When we have a static variable inside the class, **you must have declared it outside again**. So, when we declare the class variable outside the class, then it will become a global variable but we want it to be accessible only inside the Test class. So, we can use the scope resolution operator to make it accessible only inside the Test class.





Static Member Functions in C++ :- By declaring a member function as static in C++, we make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the static functions are accessed using the class name and the scope resolution operator (::) and even we can also access them using objects of the class.

Note :- the important thing is Static Member Functions in C++ can access only static data members of a class. They cannot access non-static data members. So static member functions also belong to a class.



Key Points on Static Members in C++:

- Static data members are members of a class.
- Only one instance of static members is created for the entire class and shared by all objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is created, even before the main starts.
- They can be accessed directly using the class name. •

Static member functions are functions of a class, they can be called using the class name, without creating the object of a class.

- They can access only static data members of a class; they cannot access non-static members of a class.
- It is visible only within the class, but its lifetime is the entire program.

Inner Class in C++ :-

Writing a class inside another class is called an **inner class**. It is useful only within that class. **If you are writing a class that is not useful everywhere or limited to one class.** Then you can write a class inside of another one.

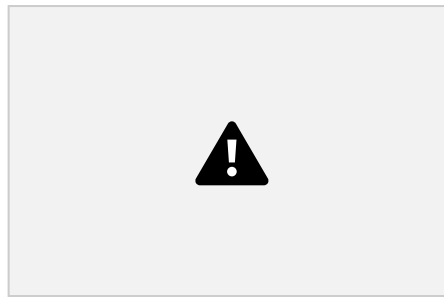
So, it is just like reducing the complexity of a bigger class, we can write smaller classes inside.



Note :- Can we create this object before the declaration of the Inner class? No, it must be done after the definition.

- Inner class can access the members of the Outer class if they are static.
- Can the Outer class create the object of the Inner class? Yes, it can. Now using that object can it access all the members of a class - We can access only those members which are public. We cannot access private and protected members of the Inner class.

When to use Inner Classes in C++?





Exception Handling in C++ :-

There are 3 types of errors that are occurred in programming:-

- Syntax Error
- Logical Error
- Runtime Error

Syntax Error :- What is a Syntax error? While typing the program, if the programmer mistypes something or did not write something properly then there is an error known as Syntax Error. So,

who will identify this error? The compiler will identify this error.

Logical Error :- Suppose you wanted to do something and so wrote the procedure or function or some code but when you run the program the results are different i.e. not as expected.

Then is there any tool for solving this type of problem? Yes, that is a debugger. The debugger will help you to run the program line by line or statement by statement, so in each statement, you can see what is happening and wherever it is going wrong and you can catch it and you can solve that problem.

Note :- The most important thing is that these two types of errors (Syntax Error And Logical Error) are faced by programmers.

Runtime Error :- Suppose you are running some business and you wanted me to develop an application for your business. So, I developed an application and that application is perfect. It is neither having any syntax errors nor having any logical errors. Then I delivered the software to you and you are a user and using that application for your business.

So, the user is going to use the application and while using the application he/she must use it properly. **If he/she mishandled the application then it may cause errors during the runtime of a program.** It's not development time. **At the development time, as a programmer, I have faced logical and syntax errors that I have already removed.** Now it is perfect. There are no errors. And you are using the program. So, at runtime, you are facing errors.

What Could Be the Cause of Error During Runtime?

- First of the reasons for runtime errors is bad input.
- Second, the program that I have given you requires an internet connection

but if you are not providing an internet connection then the program cannot continue.

- Third, the program that I have given to you needs some files on your computer system but you have deleted those files or my program is using a printer but you do not have a printer. So, these are the list of problems. **These are nothing but the unavailability of resources.**

All the things that are outside the program or not in the control of the program. Those are in the control of the user.

What happens in the case of runtime errors?

In case of runtime errors, the program crashes or program stops abnormally without completing its execution. Suddenly the program will stop, that's how the runtime errors are dangerous for the user. So, who is responsible? The user is responsible because as a developer I did my job perfectly and the user is not providing proper resources or proper input so the program is failing. Now let us give you some real-life examples.

What can we call these runtime errors?

These runtime errors are called exceptions. Why we are using the term exception? Suppose I have given you a perfect program and it will always run perfectly except in some conditions. Those conditions are giving a bad input or not providing resources. So, the program will not run in those cases. That's why we call these exceptional cases or exceptions.

What are Exceptions?

An exception is nothing but a situation in which we get the runtime error. Let us see in automobile engineering. If there is no

fuel in the car then the car will indicate that there is less fuel or it is in a reserved condition so you must find the closest fuel station or gas station.

In the same way, the programs should also behave like that. If there is bad input then the program should not stop suddenly. It should ask the user that **'this input is not proper please enter another type of Input and try again'**. If there is no internet connection then the program should not stop suddenly but it should give a prompt to the user that **"we found that there is no internet connection please connect to the internet and run it again"**. So, **the idea is the program should not terminate under such a situation but guide the user to solve the problem.**

Because who is there to resolve it? The programmer will not come to the client-side and check for the errors and remove them. The program itself can give a proper and meaningful message to the user so that users can remove this type of problem.

Note :- Giving a proper message to the user and informing them about the exact problem. And also providing him guidance to solve that problem, that's it. This is the objective of exception handling.

How to Implement Exception Handling in C++?



Inside the try block, if there is any error then it will jump to the catch block and execute the statements inside the catch block.

- **try**: It represents a block of code that can throw an exception.
- **catch**: It represents a block of code that is executed when a particular exception is thrown.
- **throw**: It is used to throw an exception inside a try block



Throwing Exceptions from C++ constructors :-

An exception should be thrown from a C++ constructor whenever an

object cannot be properly constructed or initialized. Since there is no way to recover from failed object construction, an exception should be thrown in such cases.

Since C++ constructors do not have a return type, it is **not possible to use return in the codes**. Therefore, the best practice is for constructors to throw an exception to signal failure.



Key Points regarding exception handling :-

- An exception in C++ is thrown by using the throw keyword from inside the try block. The throw keyword allows the programmer to define custom exceptions.
- **Multiple handlers (catch expressions) can be chained - each one**

with a different exception type. Only the handler whose argument type matches the exception type in the throw statement is executed.

- C++ does not require a finally block to make sure resources are released if an exception occurs.

Using Multiple catch blocks in C++ :-



Generic Catch Block (Catch - All) in C++ :-

The

following example contains a generic catch block to catch any uncaught errors/exceptions. catch(...) block takes care of all types of exceptions.

Note : If we write catch-all first, then all the exceptions will be handled here only. The lower catch blocks will never be executed. So, the catch-all block must be the last block.

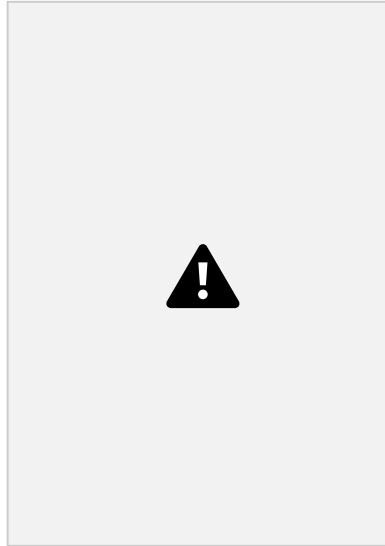
Throwing User-Defined Type in C++ :-



Can we have a Try Block inside a Try block?

Yes, we can have a try block inside another try block in C++. The following diagram shows the syntax of writing nested try blocks in

C++.



Example to Understand Try-Catch Blocks in C++:

```
class MyExp1{};
```

```
class MyExp2 : public MyExp1{};
```

So, we have these two classes. And MyExp2 is publicly inherited from the MyExp1 class. So, MyExp1 is a parent class and MyExp2 is a child class.



As you can see, we have written catch blocks for both types of exceptions. So, is this the correct format of catch block? No. We have written the catch block for the parent class first and then for the child class. **We must write catch blocks for the child class first and then for the parent class as shown in the below image.**

Inheriting Exception Class from Built-in Exception Class :-

If you are throwing your own class exception then better extend your class from the built-in exception class in C++ as follows. But it is not mandatory.



Overriding the exception class what method in C++ :-

After inheriting our custom exception class from the built-in exception class, do we have to override anything? Yes, we can override one method which is "what" method as follows. But this is not mandatory as in our previous example we have not overridden the what method and it is working fine.



How to make the function throws something in C++?





So, whatever the type of value you are throwing, you can mention that in the brackets. **And if there are more values then you can mention them with commas as follows:**





Can we use cout instead of throw?

If you use cout, the user will know the error. **And if you use throw, it will inform the calling function about the error.**

Are return and throw the same?

Return is for returning results. The throw is for reporting an error. If you change their roles then the roles of Try and Catch will also change.

Template in C++

Template in C++ allows us to define Generic Functions and Generic

Classes. That means using a Template we can implement *Generic Programming* in C++. They are going to work with a variety of data types. Templates in C++ can be represented in two ways. They are as follows.

- **Function templates**
- **Class templates**



